1.

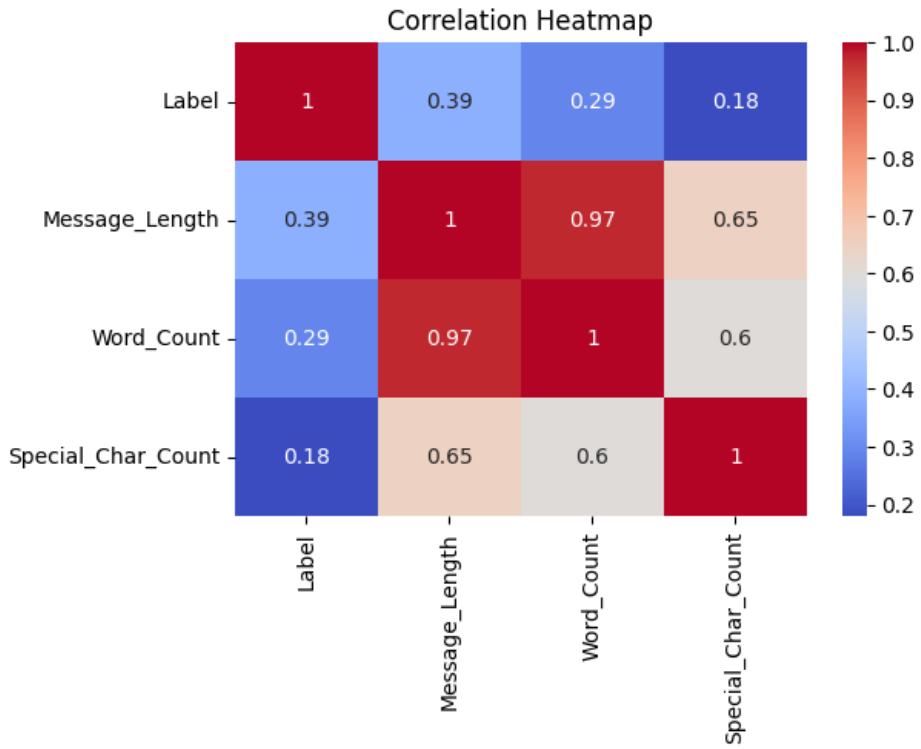
```python
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, StandardScaler
file_path = "spam.csv"
df = pd.read_csv(file_path, encoding="latin-1")
print("Original Dataset:\n", df.head())
print("\nOriginal Shape:", df.shape)
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],
        inplace=True, errors='ignore')
df.rename(columns={'v1': 'Label', 'v2': 'Message'}, inplace=True)
imputer = SimpleImputer(strategy='most_frequent')
df[['Message']] = imputer.fit_transform(df[['Message']])
label_encoder = LabelEncoder()
df['Label'] = label_encoder.fit_transform(df['Label'])
df['Message_Length'] = df['Message'].str.len()
df['Word_Count'] = df['Message'].str.split().str.len()
scaler = StandardScaler()
df[['Message_Length', 'Word_Count']] = scaler.fit_transform(
    df[['Message_Length', 'Word_Count']]
)
print("\nPreprocessed Dataset:\n", df.head())
print("\nCleaned Shape:", df.shape)
```

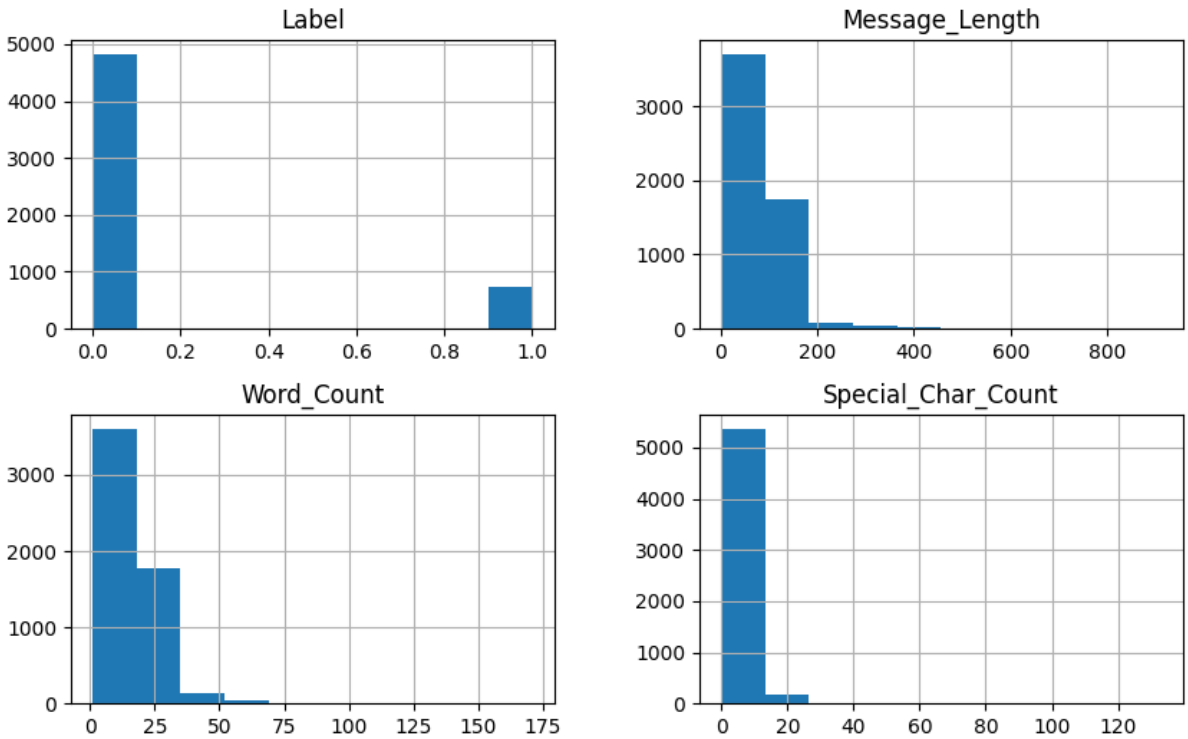
✓ 51.6s

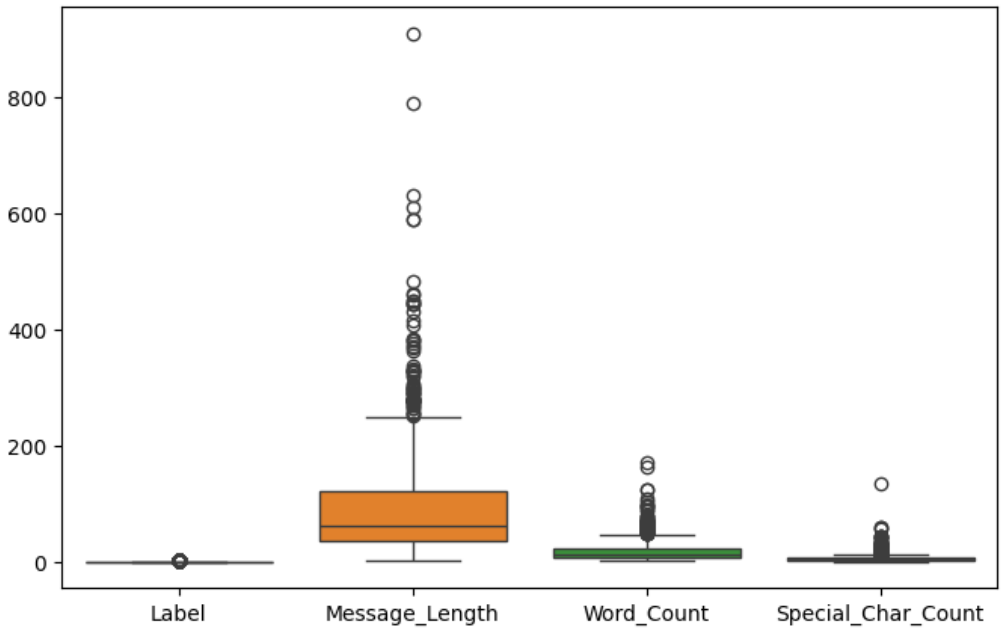
```
Original Dataset:
      v1                                                  v2 Unnamed: 2  \
0   ham  Go until jurong point, crazy.. Available only ...        NaN
1   ham                      Ok lar... Joking wif u oni...        NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...        NaN
3   ham  U dun say so early hor... U c already then say...        NaN
4   ham  Nah I don't think he goes to usf, he lives aro...        NaN

  Unnamed: 3 Unnamed: 4
0        NaN        NaN
1        NaN        NaN
2        NaN        NaN
3        NaN        NaN
4        NaN        NaN

Original Shape: (5572, 5)

Preprocessed Dataset:
    Label                                            Message  Message_Length  \
0      0  Go until jurong point, crazy.. Available only ...        0.517399
1      0                      Ok lar... Joking wif u oni...       -0.856470
2      1  Free entry in 2 a wkly comp to win FA Cup fina...        1.254596
3      0  U dun say so early hor... U c already then say...       -0.521380
4      0  Nah I don't think he goes to usf, he lives aro...       -0.320326
...
3   -0.396740
4   -0.220193

Cleaned Shape: (5572, 4)
```

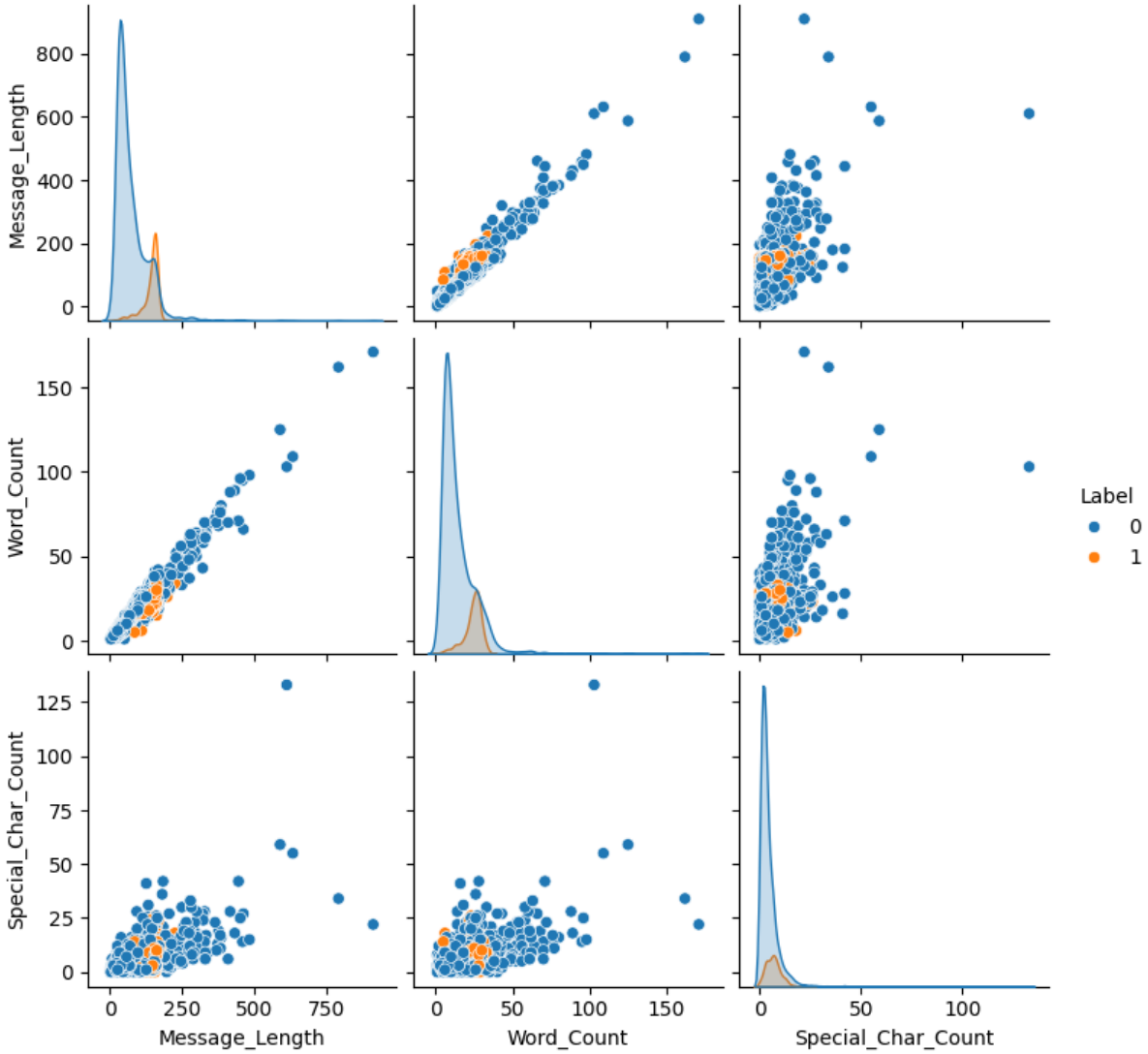
2.

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
file_path = "spam.csv"
df = pd.read_csv(file_path, encoding="latin-1")
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],
        inplace=True, errors='ignore')
df.rename(columns={'v1': 'Label', 'v2': 'Message'}, inplace=True)
df['Label'] = df['Label'].map({'ham': 0, 'spam': 1})
df['Message_Length'] = df['Message'].str.len()
df['Word_Count'] = df['Message'].str.split().str.len()
df['Special_Char_Count'] = df['Message'].str.count(r'[^A-Za-z0-9 ]')
numeric_df = df[['Label', 'Message_Length', 'Word_Count', 'Special_Char_Count']]
print("Dataset Preview:\n", df.head())
print("\nDescriptive Statistics:\n", numeric_df.describe())
corr = numeric_df.corr()
print("\nCorrelation Matrix:\n", corr)
cov = numeric_df.cov()
print("\nCovariance Matrix:\n", cov)
# Histogram
numeric_df.hist(figsize=(10, 6))
plt.show()
# Boxplot
plt.figure(figsize=(8, 5))
sns.boxplot(data=numeric_df)
plt.show()
# Heatmap for correlation
plt.figure(figsize=(6, 4))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
# Pairplot
sns.pairplot(numeric_df, hue='Label')
plt.show()
```


Correlation Heatmap
Label
Message_Length
Word_Count
Special_Char_Count
1
0.39
0.29
0.18
0.39
1
0.97
0.65
0.29
0.97
1
0.6
0.18
0.65
0.6
1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2


Label
Message_Length
Word_Count
Special_Char_Count


800
600
400
200
0
Label
Message_Length
Word_Count
Special_Char_Count


Message_Length
Word_Count
Special_Char_Count
Label
0
1
0
250
500
750
50
100
150
25
75
125

3.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.feature_extraction.text import TfidfVectorizer
file_path = "spam.csv"
df = pd.read_csv(file_path, encoding="latin-1")
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],
        inplace=True, errors='ignore')
df.rename(columns={'v1': 'Label', 'v2': 'Message'}, inplace=True)
df['Label'] = df['Label'].map({'ham': 0, 'spam': 1})
X = df['Message']
y = df['Label']
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

✓ 0.5s

```
Accuracy: 0.9425699928212491

Confusion Matrix:
 [[1199    3]
 [  77  114]]

Classification Report:
              precision    recall  f1-score   support

           0       0.94      1.00      0.97      1202
           1       0.97      0.60      0.74       191

    accuracy                           0.94      1393
   macro avg       0.96      0.80      0.85      1393
weighted avg       0.94      0.94      0.94      1393
```

4.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighb Loading... KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix
file_path = "spam.csv"
df = pd.read_csv(file_path, encoding="latin-1")
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],
        inplace=True, errors='ignore')
df.rename(columns={'v1': 'Label', 'v2': 'Message'}, inplace=True)
df['Label'] = df['Label'].map({'ham': 0, 'spam': 1})
X = df['Message']
y = df['Label']
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(X)
scaler = StandardScaler(with_mean=False)
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Train k-NN model (k = 5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
# Predictions
y_pred = knn.predict(X_test)
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

0.2s

```
Accuracy: 0.873653984206748

Confusion Matrix:
 [[1202    0]
 [ 176   15]]
```